```
import itertools
import numpy as np
```

```
def bipolar_activation(net):
  return 1 if (net >= 0) else -1
```

```
def unipolar_activation(net):
  return 1 if (net >= 0) else 0
```

```
def predict(X, weights, type):
  net = np.dot(X, weights)
  if type == 'unipolar':
    return unipolar_activation(net)
  elif type == 'bipolar':
    return bipolar_activation(net)
```

```
def train(X, y, weights, type, epochs=500, c=0.5):
  for epoch in range(epochs):
    loss = 0
    for Xi, yi in zip(X, y):
      y_pred = predict(Xi, weights, type)
      r = yi - y_pred
      loss += abs(r)
      delta_w = c*r*Xi
      weights += delta_w

    print(f'Weights after epoch {epoch}: {weights}')
    if loss == 0:
      break

  print('Learned weights : ', weights)
  weights = weights.reshape((n + 1, 1))
  test(X, y, weights, type)
```

```
def test(X, y, learned_weights, type):
  nets = np.dot(X,learned_weights).flatten()
  print('Actual Values : ',y)
  if type == 'unipolar':
    y_pred = np.array([1 if net >=0 else 0 for net in nets])
    print('Predicted Values : ', y_pred)
  else:
    y_pred = np.array([1 if net >= 0 else -1 for net in nets])
    print('Predicted Values : ', y_pred)
```

```
n = int(input('Enter number of bits : '))
X = np.array([list(i) + [1] for i in itertools.product([0,1], repeat = n)])

weights = input(f'Enter initial {n} weights and 1 bias : ')
weights = np.array([float(weight) for weight in weights.split()], dtype='longdouble')
print()
```

```
Enter number of bits : 2
Enter initial 2 weights and 1 bias : 1 1 -1.5
```

```
# 1) AND GATE UNIPOLAR
print('---- AND GATE USING PERCEPTRON ----')
y = np.array([0]*(2**n))
y[-1] = 1
train(X, y, weights.copy(), 'unipolar')
```

```
---- AND GATE USING PERCEPTRON ----
Weights after epoch 0: [ 1.   1.  -1.5]
Learned weights :  [ 1.   1.  -1.5]
Actual Values :  [0 0 0 1]
Predicted Values :  [0 0 0 1]
```

```
# 2) OR GATE UNIPOLAR
print('---- OR GATE USING PERCEPTRON ----')
y = np.array([1]*(2**n))
y[0] = 0
train(X, y, weights.copy(), 'unipolar')
```

```
---- OR GATE USING PERCEPTRON ----
Weights after epoch 0: [ 1.   1.5 -1. ]
```

```
Weights after epoch 1: [ 1.   1.5 -1. ]
Learned weights :  [ 1.   1.5 -1. ]
Actual Values :  [0 1 1 1]
Predicted Values :  [0 1 1 1]
```

```
# 3) NOR GATE UNIPOLAR
print('---- NOR GATE USING PERCEPTRON ----')
y = np.array([0]*(2**n))
y[0] = 1
train(X, y, weights.copy(), 'unipolar')
```

```
---- NOR GATE USING PERCEPTRON ----
Weights after epoch 0: [ 0.5  0.  -2. ]
Weights after epoch 1: [ 0.5  0.  -1.5]
Weights after epoch 2: [ 0.5  0.  -1. ]
Weights after epoch 3: [ 0.  0. -1.]
Weights after epoch 4: [ 0.   0.  -0.5]
Weights after epoch 5: [ 0.  -0.5 -0.5]
Weights after epoch 6: [-0.5 -0.5 -0.5]
Weights after epoch 7: [-0.5 -0.5  0. ]
Weights after epoch 8: [-0.5 -0.5  0. ]
Learned weights :  [-0.5 -0.5  0. ]
Actual Values :  [1 0 0 0]
Predicted Values :  [1 0 0 0]
```

```
# 4) NAND GATE UNIPOLAR
print('---- NAND GATE USING PERCEPTRON ----')
y = np.array([1]*(2**n))
y[-1] = 0
train(X, y, weights.copy(), 'unipolar')
```

```
---- NAND GATE USING PERCEPTRON ----
Weights after epoch 0: [ 0.5  0.5 -1.5]
Weights after epoch 1: [ 0.   0.5 -1. ]
Weights after epoch 2: [ 0.   0.  -0.5]
Weights after epoch 3: [-0.5 -0.5 -0.5]
Weights after epoch 4: [-1.  -0.5  0. ]
Weights after epoch 5: [-1.  -0.5  0.5]
Weights after epoch 6: [-1.  -1.   0.5]
Weights after epoch 7: [-1.  -0.5  1. ]
Weights after epoch 8: [-1.  -0.5  1. ]
Learned weights :  [-1.  -0.5  1. ]
Actual Values :  [1 1 1 0]
Predicted Values :  [1 1 1 0]
```

```
# 5) AND GATE BIPOLAR
print('---- AND GATE USING PERCEPTRON ----')
y = np.array([-1]*(2**n))
y[-1] = 1
train(X, y, weights.copy(), 'bipolar')
```

```
---- AND GATE USING PERCEPTRON ----
Weights after epoch 0: [ 1.   1.  -1.5]
Learned weights :  [ 1.   1.  -1.5]
Actual Values :  [-1 -1 -1  1]
Predicted Values :  [-1 -1 -1  1]
```

```
# 6) OR GATE BIPOLAR
print('---- OR GATE USING PERCEPTRON ----')
y = np.array([1]*(2**n))
y[0] = -1
train(X, y, weights.copy(), 'bipolar')
```

```
---- OR GATE USING PERCEPTRON ----
Weights after epoch 0: [ 1.   2.  -0.5]
Weights after epoch 1: [ 1.   2.  -0.5]
Learned weights :  [ 1.   2.  -0.5]
Actual Values :  [-1  1  1  1]
Predicted Values :  [-1  1  1  1]
```

```
# 7) NOR GATE BIPOLAR
print('---- NOR GATE USING PERCEPTRON ----')
y = np.array([-1]*(2**n))
y[0] = 1
train(X, y, weights.copy(), 'bipolar')
```

```
---- NOR GATE USING PERCEPTRON ----
Weights after epoch 0: [ 1.   0.  -1.5]
Weights after epoch 1: [ 0.   0.  -1.5]
Weights after epoch 2: [ 0.   0.  -0.5]
Weights after epoch 3: [ 0.  -1.  -0.5]
Weights after epoch 4: [-1.  -1.  -0.5]
Weights after epoch 5: [-1.  -1.   0.5]
```

```
Weights after epoch 6: [-1.  -1.   0.5]
Learned weights :  [-1.  -1.   0.5]
Actual Values :  [ 1 -1 -1 -1]
Predicted Values :  [ 1 -1 -1 -1]
```

```
# 8) NAND GATE BIPOLAR
print('---- NAND GATE USING PERCEPTRON ----')
y = np.array([1]*(2**n))
y[-1] = -1
train(X, y, weights.copy(), 'bipolar')
```

```
---- NAND GATE USING PERCEPTRON ----
Weights after epoch 0: [ 0.   0.  -1.5]
Weights after epoch 1: [-1.   0.  -0.5]
Weights after epoch 2: [-1.  -1.   0.5]
Weights after epoch 3: [-2.  -1.   0.5]
Weights after epoch 4: [-2.  -1.   1.5]
Weights after epoch 5: [-2.  -2.   1.5]
Weights after epoch 6: [-2.  -1.   2.5]
Weights after epoch 7: [-2.  -1.   2.5]
Learned weights :  [-2.  -1.   2.5]
Actual Values :  [ 1  1  1 -1]
Predicted Values :  [ 1  1  1 -1]
```

```
import itertools
import numpy as np
```

```
def bipolar(net, lambdaa = 1):
  return (2/(1+np.exp(-lambdaa*net))) - 1
```

```
def unipolar(net, lambdaa = 0.3):
  return (1/(1+np.exp(-lambdaa*net)))
```

```
def bipolar_fdash(y_pred):
  return (1/2)*(1-(y_pred**2))
```

```
def unipolar_fdash(y_pred):
  return (y_pred*(1-y_pred))
```

```
def predict(X, weights, type):
    net = np.dot(X, weights)
    if type == 'unipolar':
        return unipolar(net)
    elif type == 'bipolar':
        return bipolar(net)

def calculate_r(yi, y_pred, type):
    r = yi - y_pred
    if type == 'unipolar':
        return r * unipolar_fdash(y_pred)
    elif type == 'bipolar':
        return r * bipolar_fdash(y_pred)
```

```
def train(X, y, weights, type, epochs=500, c=0.5):
    for epoch in range(epochs):
        loss = 0
        for Xi, yi in zip(X, y):
            y_pred = predict(Xi, weights, type)
            r = calculate_r(yi, y_pred, type)
            loss += abs(r)
            delta_w = c*r*Xi
            weights += delta_w

        if (epoch + 1) % 50 == 0:
            print(f"Weights after epoch {epoch + 1}: ", weights)
        if loss == 0:
            break

    print('Learned Weights: ', weights)
    weights = weights.reshape((n+1, 1))
    test(X, y, weights, type)
```

```
def test(X, y, learned_weights, type):
    nets = np.dot(X, learned_weights).flatten()
    print('Actual values: ', y)

    if type == 'unipolar':
        y_pred = np.array([unipolar(net) for net in nets])
        print("Predicted values: ", y_pred)
    else:
        y_pred = np.array([bipolar(net) for net in nets])
        print("Predicted values: ", y_pred)
```

```
n = int(input("Enter the number of bits: "))
X = np.array([list(i) + [1] for i in itertools.product([0,1], repeat=n)])

weights = input(f'Enter initial {n} weights and 1 bias : ')
weights = np.array([float(weight) for weight in weights.split()], dtype='longdouble')
```

```
Enter the number of bits: 2
Enter initial 2 weights and 1 bias : 1 1 -1.5
```

```
# 1) AND GATE UNIPOLAR
print('---- AND GATE USING PERCEPTRON ----')
y = np.array([0]*(2**n))
y[-1] = 1
train(X, y, weights.copy(), 'unipolar')
```

```
---- AND GATE USING PERCEPTRON ----
Weights after epoch 50:  [ 2.06884716  2.04969855 -4.15009132]
Weights after epoch 100:  [ 3.21454526  3.18761228 -5.59950151]
Weights after epoch 150:  [ 4.12101034  4.09107424 -6.8015584 ]
Weights after epoch 200:  [ 4.86924412  4.83886183 -7.83869085]
Weights after epoch 250:  [ 5.51133257  5.48178246 -8.75087868]
Weights after epoch 300:  [ 6.07637942  6.04823695 -9.56539652]
Weights after epoch 350:  [  6.58209923   6.55557193 -10.30114698]
Weights after epoch 400:  [  7.04019002   7.01530007 -10.97171574]
Weights after epoch 450:  [  7.45889716   7.43557807 -11.587263  ]
Weights after epoch 500:  [  7.84433339   7.82248044 -12.1556508 ]
Learned Weights:  [  7.84433339   7.82248044 -12.1556508 ]
Actual values:  [0 0 0 1]
Predicted values:  [0.02541443 0.21417325 0.21527869 0.74141746]
```

```
# 2) OR GATE UNIPOLAR
print('---- OR GATE USING PERCEPTRON ----')
y = np.array([1]*(2**n))
y[0] = 0
train(X, y, weights.copy(), 'unipolar')
```

```
---- OR GATE USING PERCEPTRON ----
Weights after epoch 50:  [ 3.63909071  3.6534463  -0.01317792]
Weights after epoch 100:  [ 4.85431882  4.87106131 -0.68777983]
Weights after epoch 150:  [ 5.83760728  5.85484642 -1.36782382]
Weights after epoch 200:  [ 6.68438769  6.70133652 -1.93282344]
Weights after epoch 250:  [ 7.42466934  7.44093767 -2.40177081]
Weights after epoch 300:  [ 8.07798159  8.09339764 -2.79928944]
Weights after epoch 350:  [ 8.65953452  8.67404876 -3.14299425]
Weights after epoch 400:  [ 9.1814258   9.19505335 -3.44493946]
Weights after epoch 450:  [ 9.65329659  9.6660841  -3.71359625]
Weights after epoch 500:  [10.08285878 10.09486608 -3.95512957]
Learned Weights:  [10.08285878 10.09486608 -3.95512957]
Actual values:  [0 1 1 1]
Predicted values:  [0.23387853 0.86317574 0.86274975 0.99236068]
```

```
# 3) NOR GATE UNIPOLAR
print('---- NOR GATE USING PERCEPTRON ----')
y = np.array([0]*(2**n))
y[0] = 1
train(X, y, weights.copy(), 'unipolar')
```

```
---- NOR GATE USING PERCEPTRON ----
Weights after epoch 50:  [-1.79174464 -1.80368285 -2.14901381]
Weights after epoch 100:  [-3.15956598 -3.1744099  -0.92495224]
Weights after epoch 150:  [-4.33371745 -4.35027974  0.17307251]
Weights after epoch 200:  [-5.36795999 -5.38526488  1.00942644]
Weights after epoch 250:  [-6.27166612 -6.28891867  1.65439941]
Weights after epoch 300:  [-7.06176753 -7.07847063  2.17302379]
Weights after epoch 350:  [-7.75674595 -7.77264756  2.60510771]
Weights after epoch 400:  [-8.37291534 -8.38791913  2.97454145]
Weights after epoch 450:  [-8.92368652 -8.93778341  3.29647892]
Weights after epoch 500:  [-9.41983492 -9.4330606   3.58113845]
Learned Weights:  [-9.41983492 -9.4330606   3.58113845]
Actual values:  [1 0 0 0]
Predicted values:  [0.74542168 0.1473452  0.14784438 0.01013547]
```

```
# 4) NAND GATE UNIPOLAR
print('---- NAND GATE USING PERCEPTRON ----')
y = np.array([1]*(2**n))
y[-1] = 0
train(X, y, weights.copy(), 'unipolar')
```

```
---- NAND GATE USING PERCEPTRON ----
Weights after epoch 50:  [-0.28234401 -0.26244769  2.3176296 ]
Weights after epoch 100:  [-1.89630502 -1.86825878  4.03801651]
Weights after epoch 150:  [-3.1066732  -3.07466593  5.46235154]
Weights after epoch 200:  [-4.03497992 -4.00196397  6.68258568]
Weights after epoch 250:  [-4.79639096 -4.76405692  7.73498854]
Weights after epoch 300:  [-5.44772644 -5.41688584  8.65897372]
Weights after epoch 350:  [-6.01975913 -5.99072714  9.48285662]
Weights after epoch 400:  [-6.5310192  -6.50385404 10.22625855]
Weights after epoch 450:  [-6.99365328 -6.96828897 10.90322307]
Weights after epoch 500:  [-7.41617549 -7.3924919  11.52421072]
Learned Weights:  [-7.41617549 -7.3924919  11.52421072]
Actual values:  [1 1 1 0]
Predicted values:  [0.96944701 0.77547969 0.7742402  0.27183409]
```

```
# 5) AND GATE BIPOLAR
print('---- AND GATE USING PERCEPTRON ----')
y = np.array([-1]*(2**n))
y[-1] = 1
train(X, y, weights.copy(), 'bipolar')
```

```
---- AND GATE USING PERCEPTRON ----
Weights after epoch 50:  [ 3.0647508   3.00073714 -4.55023391]
Weights after epoch 100:  [ 3.78249802  3.74775663 -5.6901228 ]
Weights after epoch 150:  [ 4.22530613  4.20182914 -6.37837491]
Weights after epoch 200:  [ 4.54385074  4.52621902 -6.86849581]
Weights after epoch 250:  [ 4.79186665  4.77778938 -7.24789224]
Weights after epoch 300:  [ 4.99455859  4.98286282 -7.5567974 ]
Weights after epoch 350:  [ 5.16572803  5.15573565 -7.81698281]
Weights after epoch 400:  [ 5.313736    5.30502068 -8.04153109]
Weights after epoch 450:  [ 5.44402351  5.4363002  -8.23890561]
Weights after epoch 500:  [ 5.56032592  5.55339487 -8.41489048]
Learned Weights:  [ 5.56032592  5.55339487 -8.41489048]
Actual values:  [-1 -1 -1  1]
Predicted values:  [-0.99955701 -0.89181975 -0.8911083   0.87391518]
```

```
# 6) OR GATE BIPOLAR
print('---- OR GATE USING PERCEPTRON ----')
y = np.array([1]*(2**n))
y[0] = -1
train(X, y, weights.copy(), 'bipolar')
```

```
---- OR GATE USING PERCEPTRON ----
Weights after epoch 50:  [ 3.61579381  3.65075785 -1.49469801]
Weights after epoch 100:  [ 4.39603673  4.41601861 -1.91714424]
Weights after epoch 150:  [ 4.86066098  4.87445901 -2.16197481]
Weights after epoch 200:  [ 5.18925445  5.19974165 -2.3331059 ]
Weights after epoch 250:  [ 5.44258714  5.45102536 -2.46415489]
Weights after epoch 300:  [ 5.64831248  5.65536211 -2.57010427]
Weights after epoch 350:  [ 5.82127992  5.82732825 -2.65889964]
Weights after epoch 400:  [ 5.97036338  5.97565639 -2.73524918]
Weights after epoch 450:  [ 6.10127968  6.10598311 -2.80216732]
Weights after epoch 500:  [ 6.21792319  6.2221539  -2.8616978 ]
Learned Weights:  [ 6.21792319  6.2221539  -2.8616978 ]
Actual values:  [-1  1  1  1]
Predicted values:  [-0.89184043  0.93289114  0.93261621  0.99986159]
```

```
# 7) NOR GATE BIPOLAR
print('---- NOR GATE USING PERCEPTRON ----')
y = np.array([-1]*(2**n))
y[0] = 1
train(X, y, weights.copy(), 'bipolar')
```

```
---- NOR GATE USING PERCEPTRON ----
Weights after epoch 50:  [-2.97472076 -3.02599701  1.12991304]
Weights after epoch 100:  [-4.09674733 -4.12183424  1.75713199]
Weights after epoch 150:  [-4.66914899 -4.68528746  2.06152202]
Weights after epoch 200:  [-5.04938819 -5.0611983   2.26044009]
Weights after epoch 250:  [-5.33278769 -5.34207119  2.40744186]
Weights after epoch 300:  [-5.55811004 -5.56574459  2.52369852]
Weights after epoch 350:  [-5.74483009 -5.75130643  2.61968355]
Weights after epoch 400:  [-5.90408145 -5.90970085  2.70132494]
Weights after epoch 450:  [-6.04281265 -6.04777298  2.77229623]
Weights after epoch 500:  [-6.16564539 -6.17008345  2.83502778]
Learned Weights:  [-6.16564539 -6.17008345  2.83502778]
Actual values:  [ 1 -1 -1 -1]
Predicted values:  [ 0.88907914 -0.93122408 -0.93092874 -0.99985041]
```

```
# 8) NAND GATE BIPOLAR
print('---- NAND GATE USING PERCEPTRON ----')
y = np.array([1]*(2**n))
y[-1] = -1
train(X, y, weights.copy(), 'bipolar')
```

```
---- NAND GATE USING PERCEPTRON ----
Weights after epoch 50:  [-2.56383172 -2.46880693  3.73168195]
Weights after epoch 100:  [-3.53062038 -3.4873509   5.29400115]
Weights after epoch 150:  [-4.05911804 -4.03189379  6.12113385]
Weights after epoch 200:  [-4.42052894 -4.4008221   6.67916638]
Weights after epoch 250:  [-4.69411708 -4.67873006  7.0985642 ]
Weights after epoch 300:  [-4.91373963 -4.90114501  7.4337404 ]
Weights after epoch 350:  [-5.0969204  -5.08627428  7.71246036]
Weights after epoch 400:  [-5.25387996 -5.24466846  7.95076581]
Weights after epoch 450:  [-5.39108948 -5.38297711  8.15874559]
Weights after epoch 500:  [-5.51290024 -5.50565615  8.34314935]
Learned Weights:  [-5.51290024 -5.50565615  8.34314935]
Actual values:  [ 1  1  1 -1]
Predicted values:  [ 0.99952407  0.88933716  0.88857742 -0.87111956]
```

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
```

```
def predict(X, weights, lambdaa = 1):
  net = np.dot(X, weights)
  return (1 / (1 + np.exp(-lambdaa * net)))

def calculate_dw(X, y, y_pred):
  return ((y_pred - y) * (y_pred) * (1 - y_pred) * X)

def calculate_error(y, y_pred):
  return np.square(y_pred - y)
```

```
def train_vanilla(X, y, weights, epochs = 500, lr = 0.001, gamma = 0.9):
    upd = 0
    for epoch in range(epochs):
        dw = 0
        error = 0

        for (Xi, yi) in zip(X, y):
            y_pred = predict(Xi, weights)
            dw += calculate_dw(Xi, yi, y_pred)
            error += calculate_error(yi, y_pred)

        upd = (lr * dw) + (gamma * upd)
        weights -= upd
        error /= (2 * len(X))

        if((epoch + 1) % 50 == 0):
            print(f"Weights after {epoch + 1} epochs: {weights} \n")
            print(f"Error after {epoch + 1} epochs: {error} \n")
```

```
def train_stochastic(X, y, weights, epochs = 500, lr = 0.001, gamma = 0.9):
    upd = 0

    for epoch in range(epochs):
        dw = 0
        error = 0

        for (Xi, yi) in zip(X, y):
            y_pred = predict(Xi, weights)
            dw += calculate_dw(Xi, yi, y_pred)
            error += calculate_error(yi, y_pred)
            upd = (lr * dw) + (gamma * upd)
            weights -= upd

        error /= (2 * len(X))

        if((epoch + 1) % 50 == 0):
            print(f"Weights after {epoch + 1} epochs: {weights} \n")
            print(f"Error after {epoch + 1} epochs: {error} \n")
```

```
def train_minibatch(X, y, weights, epochs = 500, lr = 0.001, gamma = 0.9, bs = 32):
    upd = 0

    for epoch in range(epochs):
        dw = 0
        error = 0
        i = 0

        for (Xi, yi) in zip(X, y):
            y_pred = predict(Xi, weights)
            dw += calculate_dw(Xi, yi, y_pred)
            i += 1
            error += calculate_error(yi, y_pred)

            if i % bs == 0 or i == len(X):
                upd = (lr * dw) + (gamma * upd)
                weights -= upd
                dw = 0

        error /= (2 * len(X))

        if((epoch + 1) % 100 == 1):
            print(f"Weights after {epoch + 1} epochs: {weights} \n")
            print(f"Error after {epoch + 1} epochs: {error} \n")
```

+ Code + Text

```
df = pd.read_csv('bank_note.csv')
df.insert(4, 'x0', 1)
```

```
X = df[['variance', 'skewness', 'curtosis', 'entropy', 'x0']].values
y = df['class'].values
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=13, test_size = 0.2)
```

```
weights = input("Enter 4 weights and 1 bias: ").split()
weights = [float(weight) for weight in weights]
weights = np.array(weights)
```

```
print('Momentum GD (Vanilla)')
train_vanilla(X_train,y_train,weights.copy())
```

```
Momentum GD (Vanilla)
Weights after 50 epochs: [-4.39170985 -2.71569597 -3.0187527  -0.86156677  2.25821314]

Error after 50 epochs: 0.00608852479752056

Weights after 100 epochs: [-4.3363496  -2.3855687  -2.90810058 -0.5306964   3.26719043]

Error after 100 epochs: 0.003856002225942166

Weights after 150 epochs: [-4.28487923 -2.24673616 -2.81282294 -0.33574806  3.66350794]

Error after 150 epochs: 0.003475770487716895

Weights after 200 epochs: [-4.2521603  -2.19706772 -2.77880628 -0.25170574  3.86218869]

Error after 200 epochs: 0.0033891432638743723

Weights after 250 epochs: [-4.24795409 -2.18035801 -2.7705267  -0.21220897  3.96888747]

Error after 250 epochs: 0.0033664463651602423

Weights after 300 epochs: [-4.25985254 -2.17837356 -2.77476202 -0.19318412  4.03166804]

Error after 300 epochs: 0.0033586806182746218

Weights after 350 epochs: [-4.27904658 -2.18354698 -2.78513324 -0.1844168   4.07337171]

Error after 350 epochs: 0.0033545529592607644

Weights after 400 epochs: [-4.30131702 -2.19209819 -2.79833045 -0.18074011  4.10465489]

Error after 400 epochs: 0.0033514225794309322

Weights after 450 epochs: [-4.3246972  -2.20214642 -2.81269619 -0.17956236  4.13056985]

Error after 450 epochs: 0.0033486556753649322

Weights after 500 epochs: [-4.34826374 -2.21277075 -2.8274194  -0.17960206  4.15355833]

Error after 500 epochs: 0.003346090902787977
```

```
print('Momentum GD (Stochastic)')
train_stochastic(X_train,y_train,weights.copy())
```

```
Momentum GD (Stochastic)
C:\Users\Om Shete\AppData\Local\Temp\ipykernel_12340\4124679790.py:3: RuntimeWarning: overflow encountered in exp
  return (1 / (1 + np.exp(-lambdaa * net)))
Weights after 50 epochs: [-75.48036473 -50.36727033 -52.66577175  -5.4652907   52.97378194]

Error after 50 epochs: 0.009115767201358094

Weights after 100 epochs: [-75.46988016 -50.33777673 -52.69960101  -5.48295222  52.9809665 ]

Error after 100 epochs: 0.009115759107938713

Weights after 150 epochs: [-75.46003206 -50.31127291 -52.73012158  -5.49847125  52.98743587]

Error after 150 epochs: 0.009115752151093162

Weights after 200 epochs: [-75.44885613 -50.28209806 -52.76381042  -5.51532484  52.99456143]

Error after 200 epochs: 0.009115743364477184

Weights after 250 epochs: [-75.43385857 -50.24360928 -52.80832058  -5.53747081  53.00395412]

Error after 250 epochs: 0.009115727767399868
```

```
Weights after 300 epochs: [-75.40763212 -50.17673894 -52.88568705  -5.57615567  53.02023362]

Error after 300 epochs: 0.009115679917996871

Weights after 350 epochs: [-76.15748216 -48.07805044 -54.09633151 -11.74811181  56.47749927]

Error after 350 epochs: 0.006836649634338515

Weights after 400 epochs: [-78.88656793 -46.45063538 -53.06228899 -11.10489503  59.54460416]

Error after 400 epochs: 0.006836387673347274

Weights after 450 epochs: [-81.24619725 -45.793152   -51.18007778 -14.82990401  64.00280858]

Error after 450 epochs: 0.006844288173019668

Weights after 500 epochs: [-81.09465203 -45.80564705 -51.35447664 -14.68203285  64.08481472]

Error after 500 epochs: 0.006838394305651929
```

```
print('Momentum GD (Mini Batch)')
train_minibatch(X_train,y_train,weights.copy())
```

```
Momentum GD (Mini Batch)
Weights after 1 epochs: [-0.33463829 -0.39171181  0.4075694  -0.48639346  0.13469183]

Error after 1 epochs: 0.18023241371182047

Weights after 101 epochs: [-2.06093861 -1.15894496 -1.40096011 -0.07785838  2.29162845]

Error after 101 epochs: 0.0044560383270496945

Weights after 201 epochs: [-2.48459274 -1.35548264 -1.66769754 -0.0891239   2.65353428]

Error after 201 epochs: 0.004068569875714 75

Weights after 301 epochs: [-2.77375195 -1.48673327 -1.84654205 -0.1014073   2.881557  ]

Error after 301 epochs: 0.0039017539823356716

Weights after 401 epochs: [-2.99681125 -1.58784422 -1.9843325  -0.11179156  3.05617612]

Error after 401 epochs: 0.0038032570740688535
```

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
```

```
def predict(X, weights, lambdaa = 1):
    net = np.dot(X, weights)
    return (1 / (1 + np.exp(-lambdaa * net)))

def calculate_dw(X, y, y_pred):
    return ((y_pred - y) * (y_pred) * (1 - y_pred) * X)

def calculate_error(yi, y_pred):
    return np.square(y_pred - yi)
```

```
def train_vanilla(X, y, weights, epochs=500, lr=0.001, gamma=0.9):
    vel = 0
    for epoch in range(epochs):
        dw = 0
        error = 0
        w_lookahead = weights - (gamma * vel)

        for Xi, yi in zip(X, y):
            y_pred = predict(Xi, w_lookahead)
            dw += calculate_dw(Xi, yi, y_pred)
            error += calculate_error(yi, y_pred)

        vel = (gamma * vel) + (lr * dw)
        weights -= vel
        error /= (2 * len(X))

        if (epoch + 1) % 50 == 0:
            print(f"Weights after epoch {epoch + 1}: ", weights)
            print(f"Error after epoch {epoch + 1}: ", error)
```

```
def train_stochastic(X, y, weights, epochs=500, lr=0.001, gamma=0.9):
    vel = 0
    for epoch in range(epochs):
        error = 0

        for Xi, yi in zip(X, y):
            w_lookahead = weights - (gamma * vel)
            y_pred = predict(Xi, w_lookahead)
            dw = calculate_dw(Xi, yi, y_pred)
            error += calculate_error(yi, y_pred)
            vel = (gamma * vel) + (lr * dw)
            weights -= vel

        error /= (2 * len(X))

        if (epoch + 1) % 50 == 0:
            print(f"Weights after epoch {epoch + 1}: ", weights)
            print(f"Error after epoch {epoch + 1}: ", error)
```

```
def train_mini_batch(X, y, weights, epochs=500, lr=0.001, gamma=0.9, bs=32):
    vel = 0
    for epoch in range(epochs):
        dw = 0
        error = 0
        i = 0
        w_lookahead = weights - (gamma * vel)

        for Xi, yi in zip(X, y):
            y_pred = predict(Xi, w_lookahead)
            dw += calculate_dw(Xi, yi, y_pred)
            i += 1
            error += calculate_error(yi, y_pred)

            if i % bs == 0 or i == len(X):
                vel = (gamma * vel) + (lr *dw)
                weights -= vel
                dw = 0
                w_lookahead = weights - (gamma * vel)

        error /= (2 * len(X))

        if (epoch + 1) % 50 == 0:
            print(f"Weights after epoch {epoch + 1}: ", weights)
            print(f"Error after epoch {epoch + 1}: ", error)
```

```
df = pd.read_csv('bank_note.csv')
df.insert(4, 'x0', 1)
```

```
X = df[['variance', 'skewness', 'curtosis', 'entropy', 'x0']].values
y = df['class']
```

```
X_train, y_train, X_test, y_test = train_test_split(X, y, random_state=13, test_size=0.2)

weights = input('Enter 4 weights and 1 bias: ').split()
weights = np.array([float(weight) for weight in weights], dtype='longdouble')
```

```
print('Nesterov Accelerator GD (Vanilla)')
train_vanilla(X, y, weights.copy())
```

```
Nesterov Accelerator GD (Vanilla)
Weights after epoch 50:  [-3.72513351 -2.23946531 -2.63124085 -0.64636901  2.68248116]
Error after epoch 50:  0.004082919528475229
Weights after epoch 100:  [-3.70683594 -2.06163838 -2.53118121 -0.32000033  3.39726197]
Error after epoch 100:  0.003236919062077093
Weights after epoch 150:  [-3.75047664 -2.05563292 -2.54339637 -0.2500227   3.57421538]
Error after epoch 150:  0.0031861826133179312
Weights after epoch 200:  [-3.81428895 -2.08400725 -2.58355819 -0.24260628  3.65796628]
Error after epoch 200:  0.0031667937587036973
Weights after epoch 250:  [-3.8816543  -2.11606703 -2.62656328 -0.24485354  3.71976416]
Error after epoch 250:  0.003150571844462434
Weights after epoch 300:  [-3.94678231 -2.1476307  -2.66840945 -0.24911844  3.77462285]
Error after epoch 300:  0.0031361003778591448
Weights after epoch 350:  [-4.0088642  -2.17785476 -2.70837022 -0.25364241  3.82587144]
Error after epoch 350:  0.0031230752071045153
Weights after epoch 400:  [-4.06795299 -2.20666477 -2.74643407 -0.25804637  3.87445563]
Error after epoch 400:  0.0031112902512163065
Weights after epoch 450:  [-4.12426459 -2.23414122 -2.78272806 -0.26225675  3.92075955]
Error after epoch 450:  0.0031005790737727895
Weights after epoch 500:  [-4.17802703 -2.26038773 -2.81739453 -0.26626944  3.96501348]
Error after epoch 500:  0.0030908041894093176
```

```
print('Nesterov Accelerator GD (Stochastic)')
train_stochastic(X, y, weights.copy())
```

```
Nesterov Accelerator GD (Stochastic)
Weights after epoch 50:  [-1.78832473 -1.09499963 -1.26125843 -0.14033199  2.02572752]
Error after epoch 50:  0.004760527334860335
Weights after epoch 100:  [-2.17715925 -1.30631838 -1.52699482 -0.1494569   2.39643232]
Error after epoch 100:  0.004164190815010794
Weights after epoch 150:  [-2.44794548 -1.44111437 -1.70270582 -0.16200781  2.62019199]
Error after epoch 150:  0.0039183583116630194
Weights after epoch 200:  [-2.66097815 -1.54313988 -1.83813856 -0.17274211  2.78932972]
Error after epoch 200:  0.003772946841441464
Weights after epoch 250:  [-2.83878806 -1.62685964 -1.95026783 -0.18186525  2.92894738]
Error after epoch 250:  0.003672996610109214
Weights after epoch 300:  [-2.99251485 -1.69872018 -2.04693725 -0.18980778  3.04950268]
Error after epoch 300:  0.0035984529795541277
Weights after epoch 350:  [-3.12855848 -1.76215124 -2.13244623 -0.19687422  3.15643951]
Error after epoch 350:  0.0035399521108837755
Weights after epoch 400:  [-3.2509671  -1.81920857 -2.20943049 -0.20326677  3.25300568]
Error after epoch 400:  0.0034924180692374436
Weights after epoch 450:  [-3.36248069 -1.8712327  -2.27963754 -0.20912309  3.34132333]
Error after epoch 450:  0.0034528087567235943
Weights after epoch 500:  [-3.46504975 -1.91915347 -2.34429458 -0.21454059  3.42287141]
Error after epoch 500:  0.0034191647822317196
```

```
print('Nesterov Accelerator GD (Mini Batch)')
train_mini_batch(X, y, weights.copy())
```

```
Nesterov Accelerator GD (Mini Batch)
Weights after epoch 50:  [-1.84011053 -1.10843593 -1.2627129  -0.12812667  2.0187505 ]
Error after epoch 50:  0.0048066605413041
Weights after epoch 100:  [-2.22288091 -1.30657444 -1.5228339  -0.13360726  2.389217  ]
Error after epoch 100:  0.004204829115502439
Weights after epoch 150:  [-2.49111634 -1.43863996 -1.69813733 -0.14675294  2.61163658]
Error after epoch 150:  0.003948123142184919
Weights after epoch 200:  [-2.70236952 -1.54007112 -1.83385998 -0.15871609  2.77974934]
Error after epoch 200:  0.0037950142843448044
Weights after epoch 250:  [-2.87867019 -1.62377356 -1.94631142 -0.16913731  2.91847547]
Error after epoch 250:  0.003689725184325798
Weights after epoch 300:  [-3.03099532 -1.6957692  -2.04320772 -0.1783176   3.03816071]
Error after epoch 300:  0.0036113548987655045
Weights after epoch 350:  [-3.1656769  -1.7593471  -2.12882964 -0.18652829  3.14420443]
Error after epoch 350:  0.0035500126203688092
Weights after epoch 400:  [-3.28673202 -1.81651212 -2.20581969 -0.19396604  3.2398401 ]
Error after epoch 400:  0.003500301918335255
Weights after epoch 450:  [-3.39688641 -1.86858717 -2.27593788 -0.20077234  3.32718801]
Error after epoch 450:  0.0034589800029814284
```

```
Weights after epoch 500:  [-3.49808467 -1.91649815 -2.34042416 -0.20705164  3.4077303 ]
Error after epoch 500:  0.003423957227593001
```

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
```

```
def predict(X, weights, lambdaa = 1):
    net = np.dot(X, weights)
    return (1 / (1 + np.exp(-lambdaa * net)))

def calculate_dw(X, y, y_pred):
    return ((y_pred - y) * (y_pred) * (1 - y_pred) * X)

def calculate_error(yi, y_pred):
    return np.square(y_pred - yi)
```

```
def train_vanilla(X, y, weights, epochs=500, lr=0.01, epislon=1e-8):
    vel = 0
    for epoch in range(epochs):
        error = 0
        dw = 0
        for Xi, yi in zip(X, y):
            y_pred = predict(Xi, weights)
            dw += calculate_dw(Xi, yi, y_pred)
            error += calculate_error(yi, y_pred)

        vel = vel + (dw**2)
        weights -= ((lr * dw) / (np.sqrt(vel + epislon)))
        error /= (2 * len(X))

        if (epoch+1) % 50 == 0:
            print(f'Weights after epoch {epoch+1} : ',weights)
            print(f'Error after epoch {epoch+1} : ',error)
```

```
def train_stochastic(X, y, weights, epochs=500, lr=0.01, epislon=1e-8):
    vel = 0
    for epoch in range(epochs):
        error = 0
        for Xi, yi in zip(X, y):
            y_pred = predict(Xi, weights)
            dw = calculate_dw(Xi, yi, y_pred)
            error += calculate_error(yi, y_pred)
            vel = vel + (dw**2)
            weights -= ((lr * dw) / (np.sqrt(vel + epislon)))

        error /= (2 * len(X))

        if (epoch+1) % 50 == 0:
            print(f'Weights after epoch {epoch+1} : ',weights)
            print(f'Error after epoch {epoch+1} : ',error)
```

```
def train_min_batch(X, y, weights, epochs=500, lr=0.01, epislon=1e-8, bs=32):
    vel = 0
    for epoch in range(epochs):
        error = 0
        i = 0
        dw = 0
        for Xi, yi in zip(X, y):
            y_pred = predict(Xi, weights)
            dw += calculate_dw(Xi, yi, y_pred)
            i += 1
            error += calculate_error(yi, y_pred)

            if i % bs == 0 or i == len(X):
                vel = vel + (dw**2)
                weights -= ((lr * dw) / (np.sqrt(vel + epislon)))
                dw = 0

        error /= (2 * len(X))

        if (epoch+1) % 50 == 0:
            print(f'Weights after epoch {epoch+1} : ',weights)
            print(f'Error after epoch {epoch+1} : ',error)
```

```
df = pd.read_csv('bank_note.csv')
df.insert(4,'x0',1)
```

```
X = df[['variance','skewness','curtosis','entropy','x0']].values
y = df['class']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=13, test_size=0.2)
weights = input('Enter 4 weights and 1 bias : ').split()
weights = np.array([float(weight) for weight in weights], dtype='longdouble')
```

```
print('Adagrad (Vanilla)')
train_vanilla(X_train,y_train,weights.copy())
```

```
Adagrad (Vanilla)
Weights after epoch 50 :  [ 0.37125579 -0.30838689  0.65753559 -0.46097721  0.16952008]
Error after epoch 50 :  0.1946341730544026
Weights after epoch 100 :  [ 0.3120671  -0.34736975  0.58641092 -0.53477082  0.25766423]
Error after epoch 100 :  0.177002668515983
Weights after epoch 150 :  [ 0.26777458 -0.37490175  0.53146222 -0.58634531  0.31704715]
Error after epoch 150 :  0.16372353833392655
Weights after epoch 200 :  [ 0.2315893  -0.39743771  0.48585268 -0.6254709   0.36178188]
Error after epoch 200 :  0.153139094685064
Weights after epoch 250 :  [ 0.20068647 -0.41692467  0.4465855  -0.65643305  0.39753448]
Error after epoch 250 :  0.14444958067308356
Weights after epoch 300 :  [ 0.17355687 -0.43409854  0.41192517 -0.68160111  0.42725289]
Error after epoch 300 :  0.13715609150323957
Weights after epoch 350 :  [ 0.14928049 -0.44932643  0.38075313 -0.70243242  0.45264718]
Error after epoch 350 :  0.1309207743162474
Weights after epoch 400 :  [ 0.12724533 -0.46285154  0.35230795 -0.71988276  0.47476727]
Error after epoch 400 :  0.12550543682790888
Weights after epoch 450 :  [ 0.10701843 -0.47486452  0.32605111 -0.73460918  0.49428376]
Error after epoch 450 :  0.12073733494995528
Weights after epoch 500 :  [ 0.08827947 -0.48552444  0.30158972 -0.7470809   0.51163933]
Error after epoch 500 :  0.11648826328538676
```

```
print('Adagrad (Stochastic)')
train_stochastic(X_train,y_train,weights.copy())
```

```
Adagrad (Stochastic)
Weights after epoch 50 :  [-0.7522853  -0.42558678 -0.42320246 -0.22794774  0.5041294 ]
Error after epoch 50 :  0.016979124903761002
Weights after epoch 100 :  [-0.89326723 -0.48221969 -0.52334002 -0.15692226  0.77670195]
Error after epoch 100 :  0.01243945084095838
Weights after epoch 150 :  [-0.97274577 -0.52046054 -0.58476738 -0.11925709  0.9517575 ]
Error after epoch 150 :  0.01050620157066231
Weights after epoch 200 :  [-1.02972711 -0.54994677 -0.62966331 -0.09477389  1.07853071]
Error after epoch 200 :  0.00940011054135062
Weights after epoch 250 :  [-1.07497758 -0.57412455 -0.66527309 -0.07729239  1.17672985]
Error after epoch 250 :  0.008673333281926056
Weights after epoch 300 :  [-1.11291578 -0.59470341 -0.69490321 -0.06408599  1.25617029]
Error after epoch 300 :  0.008154378775331586
Weights after epoch 350 :  [-1.14579082 -0.61266864 -0.72034581 -0.05372172  1.32242719]
Error after epoch 350 :  0.007762463034424446
Weights after epoch 400 :  [-1.17491504 -0.62864304 -0.74268373 -0.04536077  1.37895802]
Error after epoch 400 :  0.007454275792636227
Weights after epoch 450 :  [-1.20112876 -0.6430466  -0.76262265 -0.0384739   1.42804699]
Error after epoch 450 :  0.007204408704021717
Weights after epoch 500 :  [-1.22500695 -0.65617654 -0.78064893 -0.03270802  1.47127731]
Error after epoch 500 :  0.006996924682271778
```

```
print('Adagrad (Mini Batch)')
train_min_batch(X_train,y_train,weights.copy())
```

```
Adagrad (Mini Batch)
Weights after epoch 50 :  [-0.2076353  -0.41253244 -0.06397104 -0.61871423  0.26750986]
Error after epoch 50 :  0.061492952189977766
Weights after epoch 100 :  [-0.41797584 -0.3978772  -0.28350786 -0.43557702  0.32120382]
Error after epoch 100 :  0.031569190930990476
Weights after epoch 150 :  [-0.53659205 -0.41534542 -0.364503   -0.32382931  0.47048517]
Error after epoch 150 :  0.022653973120027985
Weights after epoch 200 :  [-0.61725333 -0.43239098 -0.41186312 -0.25828953  0.60365222]
Error after epoch 200 :  0.018438925766740093
Weights after epoch 250 :  [-0.67781422 -0.44754751 -0.44742683 -0.21417796  0.71229964]
Error after epoch 250 :  0.015940292850899377
Weights after epoch 300 :  [-0.72624402 -0.46140298 -0.47663618 -0.18201622  0.80238334]
Error after epoch 300 :  0.014274154380791331
Weights after epoch 350 :  [-0.76665919 -0.47425929 -0.50169217 -0.15735871  0.8787882 ]
Error after epoch 350 :  0.013076344482321
Weights after epoch 400 :  [-0.80141551 -0.48628381 -0.52375673 -0.13777835  0.94484933]
Error after epoch 400 :  0.01216903892103128
Weights after epoch 450 :  [-0.8319722  -0.49758838 -0.54353838 -0.12181549  1.00285838]
Error after epoch 450 :  0.011454965367686204
Weights after epoch 500 :  [-0.85929057 -0.50825752 -0.56150849 -0.10853173  1.05443898]
Error after epoch 500 :  0.010876328368886553
```

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
```

```
def predict(X, weights, lambdaa = 1):
    net = np.dot(X, weights)
    return (1 / (1 + np.exp(-lambdaa * net)))

def calculate_dw(X, y, y_pred):
    return ((y_pred - y) * (y_pred) * (1 - y_pred) * X)

def calculate_error(yi, y_pred):
    return np.square(y_pred - yi)
```

```
def train_vanilla(X, y, weights, epochs=500, lr=0.01, epislon=1e-8, beta1=0.9, beta2=0.999):
    mom = 0
    vel = 0

    for epoch in range(epochs):
        dw = 0
        error = 0

        for Xi, yi in zip(X, y):
            y_pred = predict(Xi, weights)
            dw += calculate_dw(Xi, yi, y_pred)
            error += calculate_error(yi, y_pred)

        mom = (beta1 * mom) + ((1 - beta1) * dw)
        vel = (beta2 * vel) + ((1 - beta2) * (dw**2))
        step = epoch + 1
        momcap = mom / (1 - (beta1**step))
        velcap = vel / (1 - (beta2**step))
        weights -= ((lr * momcap) / (np.sqrt(velcap + epislon)))
        error /= (2 * len(X))

        if (epoch+1)%50 == 0:
            print(f'Weights after epoch {epoch+1} : ',weights)
            print(f'Error after epoch {epoch+1} : ',error)
```

```
def train_stochastic(X, y, weights, epochs=500, lr=0.01, epislon=1e-8, beta1=0.9, beta2=0.999):
    mom = 0
    vel = 0
    step = 0

    for epoch in range(epochs):
        error = 0

        for Xi, yi in zip(X, y):
            y_pred = predict(Xi, weights)
            dw = calculate_dw(Xi, yi, y_pred)
            error += calculate_error(yi, y_pred)
            mom = (beta1 * mom) + ((1 - beta1) * dw)
            vel = (beta2 * vel) + ((1 - beta2) * (dw**2))
            step += 1
            momcap = mom / (1 - (beta1**step))
            velcap = vel / (1 - (beta2**step))
            weights -= ((lr * momcap) / (np.sqrt(velcap + epislon)))


        error /= (2 * len(X))

        if (epoch+1)%50 == 0:
            print(f'Weights after epoch {epoch+1} : ',weights)
            print(f'Error after epoch {epoch+1} : ',error)
```

```
def train_mini_batch(X, y, weights, epochs=500, lr=0.01, epislon=1e-8, beta1=0.9, beta2=0.999, bs=32):
    mom = 0
    vel = 0
    step = 0

    for epoch in range(epochs):
        error = 0
        dw = 0
        i = 0

        for Xi, yi in zip(X, y):
            y_pred = predict(Xi, weights)
            dw += calculate_dw(Xi, yi, y_pred)
```

```
            i += 1
            error += calculate_error(yi, y_pred)

            if i%bs == 0 or i == len(X):
                mom = (beta1 * mom) + ((1 - beta1) * dw)
                vel = (beta2 * vel) + ((1 - beta2) * (dw**2))
                step += 1
                momcap = mom / (1 - (beta1**step))
                velcap = vel / (1 - (beta2**step))
                weights -= ((lr * momcap) / (np.sqrt(velcap + epislon)))
                dw = 0
        error /= (2 * len(X))

        if (epoch+1)%50 == 0:
            print(f'Weights after epoch {epoch+1} : ',weights)
            print(f'Error after epoch {epoch+1} : ',error)
```

```
df = pd.read_csv('bank_note.csv')
df.insert(4,'x0',1)
```

```
X = df[['variance','skewness','curtosis','entropy','x0']].values
y = df['class']
```

```
X_train, X_test, y_train, y_test = train_test_split(X,y,random_state=13,test_size=0.2)
weights = input('Enter 4 weights and 1 bias : ').split()
weights = np.array([float(weight) for weight in weights], dtype='longdouble')
```

```
print('Adam (Vanilla)')
train_vanilla(X_train,y_train,weights.copy())
```

```
Adam (Vanilla)
Weights after epoch 50 :  [ 0.01585252 -0.48583405  0.24332349 -0.80139811  0.44419424]
Error after epoch 50 :  0.10702361087191543
Weights after epoch 100 :  [-0.33940647 -0.51950831 -0.2649651  -0.71347022  0.49268423]
Error after epoch 100 :  0.04372105121775649
Weights after epoch 150 :  [-0.60203996 -0.51101469 -0.48411533 -0.33039449  0.75317172]
Error after epoch 150 :  0.01796901131387408
Weights after epoch 200 :  [-0.75989198 -0.52242694 -0.55695582 -0.15484596  1.1010441 ]
Error after epoch 200 :  0.012057094797969023
Weights after epoch 250 :  [-0.86916062 -0.54894454 -0.61449686 -0.07761004  1.30637961]
Error after epoch 250 :  0.009852392747750576
Weights after epoch 300 :  [-0.95509729 -0.58122165 -0.66634952 -0.03989542  1.45060131]
Error after epoch 300 :  0.008647057394740282
Weights after epoch 350 :  [-1.02744948 -0.61449002 -0.71374958 -0.0194097   1.56051837]
Error after epoch 350 :  0.007858963567316095
Weights after epoch 400 :  [-1.09094327 -0.64684382 -0.75722211 -0.00760367  1.64867009]
Error after epoch 400 :  0.007294099271211063
Weights after epoch 450 :  [-1.14818115e+00 -6.77675916e-01 -7.97322839e-01 -6.84304994e-04
  1.72186709e+00]
Error after epoch 450 :  0.006865654954697856
Weights after epoch 500 :  [-1.20074398 -0.70687837 -0.8345699   0.00325253  1.78424411]
Error after epoch 500 :  0.006527598316542039
```

```
print('Adam (Stochastic)')
train_stochastic(X_train,y_train,weights.copy())
```

```
Adam (Stochastic)
Weights after epoch 50 :  [-4.62802048 -2.32821612 -2.97498474  0.13987812  4.5108557 ]
Error after epoch 50 :  0.0041023592001994255
Weights after epoch 100 :  [-6.01427437 -3.02438737 -3.89074668  0.17018931  5.72789928]
Error after epoch 100 :  0.003908207602572633
Weights after epoch 150 :  [-7.02238674 -3.54279239 -4.56128574  0.16388196  6.67713625]
Error after epoch 150 :  0.003831436359544577
Weights after epoch 200 :  [-7.87584457 -3.98523274 -5.13149363  0.11117256  7.42067564]
Error after epoch 200 :  0.0038311496147869877
Weights after epoch 250 :  [-8.64890477 -4.38688    -5.6450762   0.0505421   8.06716124]
Error after epoch 250 :  0.0038401962341422864
Weights after epoch 300 :  [-9.36141378 -4.74280533 -6.09986631  0.01933647  8.71148906]
Error after epoch 300 :  0.0038063499390327674
Weights after epoch 350 :  [-10.3196006   -5.00590074  -6.35459721   0.01702268   9.74058334]
Error after epoch 350 :  0.003762037115553857
Weights after epoch 400 :  [-11.33591423  -5.25555815  -6.61864821  -0.12054972   9.97498622]
Error after epoch 400 :  0.004216756477551327
Weights after epoch 450 :  [-11.81848029  -5.50851578  -6.98745132  -0.08954771  10.89465678]
Error after epoch 450 :  0.004088674851684614
Weights after epoch 500 :  [-12.09089101  -5.78964589  -7.42790101  -0.09259638  11.39574603]
Error after epoch 500 :  0.0037500085302667745
```

```
print('Adam (Mini Batch)')
train_mini_batch(X_train,y_train,weights.copy())
```

```
Adam (Mini Batch)
Weights after epoch 50 :  [-1.85085395 -1.0331295  -1.25164453 -0.0080995   2.29785305]
Error after epoch 50 :  0.004786800573041489
Weights after epoch 100 :  [-2.47658128 -1.32479638 -1.64029842 -0.04145616  2.73736286]
Error after epoch 100 :  0.004197142863617189
Weights after epoch 150 :  [-2.90865464 -1.52125456 -1.90693674 -0.06493766  3.04716813]
Error after epoch 150 :  0.00397468176816027
Weights after epoch 200 :  [-3.22410869 -1.66793209 -2.10497454 -0.08404852  3.28316989]
Error after epoch 200 :  0.0038566579591723634
Weights after epoch 250 :  [-3.47151737 -1.78404328 -2.26141241 -0.09930185  3.47308627]
Error after epoch 250 :  0.0037845256471140316
Weights after epoch 300 :  [-3.67597074 -1.88027073 -2.39104553 -0.11164548  3.63275395]
Error after epoch 300 :  0.003736261697266815
Weights after epoch 350 :  [-3.85074087 -1.96262676 -2.50203302 -0.12190617  3.77098825]
Error after epoch 350 :  0.0037018665867128704
Weights after epoch 400 :  [-4.00358749 -2.03470825 -2.59921547 -0.13064665  3.89308398]
Error after epoch 400 :  0.003676233860679297
Weights after epoch 450 :  [-4.13945829 -2.09882647 -2.68569375 -0.13824135  4.00248278]
Error after epoch 450 :  0.0036564923357319886
Weights after epoch 500 :  [-4.26172113 -2.15655936 -2.76358244 -0.14494647  4.10156514]
Error after epoch 500 :  0.003640900281578409
```

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
```

```
def predict(X, weights, lambdaa = 1):
    net = np.dot(X, weights)
    return (1 / (1 + np.exp(-lambdaa * net)))

def calculate_dw(X, y, y_pred):
    return ((y_pred - y) * (y_pred) * (1 - y_pred) * X)

def calculate_error(yi, y_pred):
    return np.square(y_pred - yi)
```

```
def train_vanilla(X, y, weights, epochs=500, lr=0.001, epislon=1e-8, beta=0.5):
    vel = 0
    for epoch in range(epochs):
        error = 0
        dw = 0
        for Xi, yi in zip(X, y):
            y_pred = predict(Xi, weights)
            dw += calculate_dw(Xi, yi, y_pred)
            error += calculate_error(yi, y_pred)

        vel = (beta * vel) + ((1 - beta) * (dw**2))
        weights -= ((lr * dw) / (np.sqrt(vel + epislon)))
        error /= (2 * len(X))

        if (epoch+1) % 50 == 0:
            print(f'Weights after epoch {epoch+1} : ',weights)
            print(f'Error after epoch {epoch+1} : ',error)
```

```
def train_stochastic(X, y, weights, epochs=500, lr=0.001, epislon=1e-8, beta=0.5):
    vel = 0
    for epoch in range(epochs):
        error = 0
        for Xi, yi in zip(X, y):
            y_pred = predict(Xi, weights)
            dw = calculate_dw(Xi, yi, y_pred)
            error += calculate_error(yi, y_pred)
            vel = (beta * vel) + ((1 - beta) * (dw**2))
            weights -= ((lr * dw) / (np.sqrt(vel + epislon)))

        error /= (2 * len(X))

        if (epoch+1) % 50 == 0:
            print(f'Weights after epoch {epoch+1} : ',weights)
            print(f'Error after epoch {epoch+1} : ',error)
```

```
def train_mini_batch(X, y, weights, epochs=500, lr=0.001, epislon=1e-8, beta=0.5, bs=32):
    vel = 0
    for epoch in range(epochs):
        error = 0
        dw = 0
        i = 0
        for Xi, yi in zip(X, y):
            y_pred = predict(Xi, weights)
            dw += calculate_dw(Xi, yi, y_pred)
            i += 1
            error += calculate_error(yi, y_pred)

            if i%bs == 0 or i == len(X):
                vel = (beta * vel) + ((1 - beta) * (dw**2))
                weights -= ((lr * dw) / (np.sqrt(vel + epislon)))
                dw = 0

        error /= (2 * len(X))

        if (epoch+1) % 50 == 0:
            print(f'Weights after epoch {epoch+1} : ',weights)
            print(f'Error after epoch {epoch+1} : ',error)
```

```
df = pd.read_csv('bank_note.csv')
df.insert(4,'x0',1)
```

```
X = df[['variance','skewness','curtosis','entropy','x0']].values
y = df['class']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=13, test_size=0.2)
weights = input('Enter 4 weights and 1 bias : ').split()
weights = np.array([float(weight) for weight in weights], dtype='longdouble')
```

```
print('RMSPROP (Vanilla)')
train_vanilla(X_train,y_train,weights.copy())
```

```
RMSPROP (Vanilla)
Weights after epoch 50 :  [ 0.4493069  -0.25037206  0.74903664 -0.35157574  0.06198668]
Error after epoch 50 :  0.2163499406573366
Weights after epoch 100 :  [ 0.39930191 -0.29993683  0.69883838 -0.40205345  0.11442909]
Error after epoch 100 :  0.20305518777499387
Weights after epoch 150 :  [ 0.34930281 -0.34924393  0.64868769 -0.45234997  0.16504475]
Error after epoch 150 :  0.1900083577107779
Weights after epoch 200 :  [ 0.29931734 -0.39733326  0.59856894 -0.50253783  0.21538433]
Error after epoch 200 :  0.1772488916715075
Weights after epoch 250 :  [ 0.24933116 -0.40912908  0.54843979 -0.55255459  0.26544664]
Error after epoch 250 :  0.16454668072104164
Weights after epoch 300 :  [ 0.19938231 -0.42414932  0.49835217 -0.60246006  0.31540519]
Error after epoch 300 :  0.1517611330275018
Weights after epoch 350 :  [ 0.14946333 -0.4461929   0.44831095 -0.65229241  0.36532279]
Error after epoch 350 :  0.13943686248340303
Weights after epoch 400 :  [ 0.09956499 -0.47276545  0.39830524 -0.7020503   0.41520511]
Error after epoch 400 :  0.12794796446157614
Weights after epoch 450 :  [ 0.04968164 -0.50230417  0.34832318 -0.75171989  0.46505162]
Error after epoch 450 :  0.11750347212898721
Weights after epoch 500 :  [-1.92201043e-04 -5.33548721e-01  2.98354188e-01 -8.01262652e-01
  5.14854182e-01]
Error after epoch 500 :  0.10816936243524235
```

```
print('RMSProp (Stochastic)')
train_stochastic(X_train,y_train,weights.copy())
```

```
RMSProp (Stochastic)
Weights after epoch 50 :  [-2.86730008 -1.27913962 -1.75971618  0.38487542  4.40293968]
Error after epoch 50 :  0.005154824874154497
Weights after epoch 100 :  [-3.76696051 -1.72619747 -2.38285566  0.49903036  5.67723716]
Error after epoch 100 :  0.005210018111249248
Weights after epoch 150 :  [-4.5312659  -2.08855458 -2.85993662  0.54560109  6.48693452]
Error after epoch 150 :  0.005040569176895971
Weights after epoch 200 :  [-5.22718951 -2.41013561 -3.28224936  0.5766594   7.12537978]
Error after epoch 200 :  0.004839186751381252
Weights after epoch 250 :  [-5.93101704 -2.67360922 -3.66909961  0.62812689  7.66466291]
Error after epoch 250 :  0.004611839250624377
Weights after epoch 300 :  [-6.64252837 -2.90169328 -4.0308283   0.71166195  8.15551326]
Error after epoch 300 :  0.004404283069751225
Weights after epoch 350 :  [-7.34808213 -3.10411553 -4.37652138  0.80717939  8.61703642]
Error after epoch 350 :  0.0042898737027841525
Weights after epoch 400 :  [-8.01688018 -3.29691396 -4.7078343   0.89389496  9.06299148]
Error after epoch 400 :  0.004289264429920012
Weights after epoch 450 :  [-8.64235661 -3.4923312  -5.02952933  0.95850962  9.4899824 ]
Error after epoch 450 :  0.004333987126984063
Weights after epoch 500 :  [-9.24463553 -3.70439112 -5.35502286  1.00025217  9.89914275]
Error after epoch 500 :  0.004364673110561608
```

```
print('RMSProp (Mini Batch)')
train_mini_batch(X_train,y_train,weights.copy())
```

```
RMSProp (Mini Batch)
Weights after epoch 50 :  [-0.90922785 -0.4857604  -0.5070907  -0.2269912   0.63426035]
Error after epoch 50 :  0.013924555020031978
Weights after epoch 100 :  [-1.33849478 -0.70436909 -0.83546024 -0.0556999   1.48323007]
Error after epoch 100 :  0.006653368091379175
Weights after epoch 150 :  [-1.61703277e+00 -8.45753275e-01 -1.02957262e+00 -1.39539789e-03
  1.88733160e+00]
Error after epoch 150 :  0.005311757525720115
Weights after epoch 200 :  [-1.83189912 -0.94747119 -1.1682011   0.01471029  2.14003183]
Error after epoch 200 :  0.004784596523900017
Weights after epoch 250 :  [-2.01182614 -1.03352499 -1.28306475  0.01672463  2.3262747 ]
Error after epoch 250 :  0.00448527569604139
Weights after epoch 300 :  [-2.17085422 -1.10825877 -1.38315843  0.01551631  2.47755462]
Error after epoch 300 :  0.004287034729684852
Weights after epoch 350 :  [-2.31613325 -1.17447407 -1.47309208  0.01441242  2.60858229]
Error after epoch 350 :  0.004143551415122579
Weights after epoch 400 :  [-2.45195354 -1.2344343  -1.55582696  0.01399621  2.7267326 ]
Error after epoch 400 :  0.0040334653462411425
Weights after epoch 450 :  [-2.58093555 -1.28985074 -1.63333859  0.01409421  2.83596019]
Error after epoch 450 :  0.0039455612079365805
Weights after epoch 500 :  [-2.70471446 -1.34195793 -1.70696586  0.01443457  2.93862241]
Error after epoch 500 :  0.003873355292103846
```

design and train a convolutional neural network for Image classification for domain of your choice

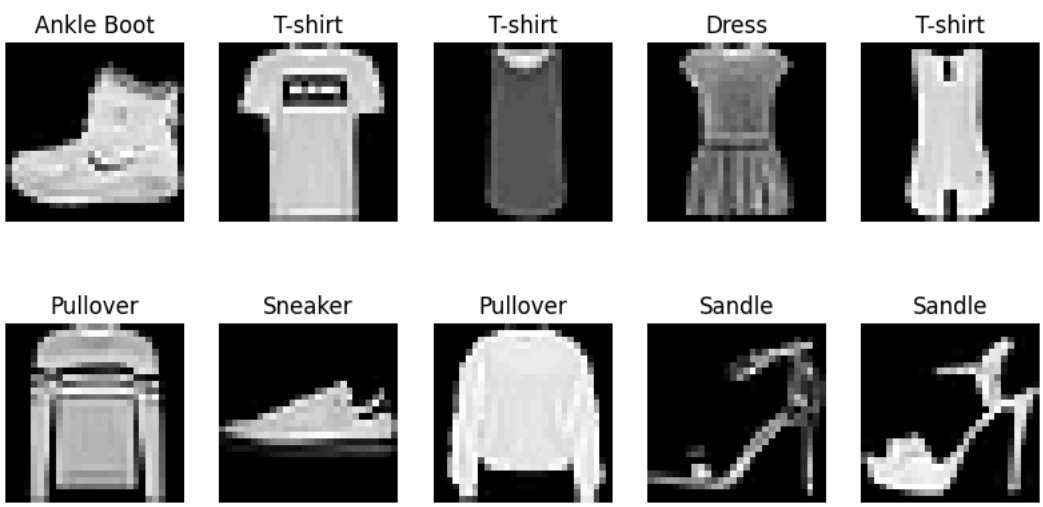
```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
```

```
(X_train, y_train), (X_test, y_test) = keras.datasets.fashion_mnist.load_data()
```

```
X_train, X_test = X_train / 255.0, X_test / 255.0

X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)

y_train = keras.utils.to_categorical(y_train, 10)
y_test = keras.utils.to_categorical(y_test, 10)
```

```
class_names = ['T-shirt', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandle', 'Shirt', 'Sneaker', 'Bag', 'Ankle Boot']
```

```
plt.figure(figsize=(10, 5))
for i in range(10):
  plt.subplot(2, 5, i+1)
  plt.imshow(X_train[i].reshape(28, 28), cmap='gray')
  plt.title(class_names[np.argmax(y_train[i])])
  plt.axis('off')
plt.show()
```




```
model = keras.Sequential([
    keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(28, 28, 1)),
    keras.layers.MaxPooling2D(2,2),
    keras.layers.Conv2D(64, (3,3), activation='relu'),
    keras.layers.MaxPooling2D(2,2),
    keras.layers.Flatten(),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dropout(0.4),
    keras.layers.Dense(10, activation='softmax')
])
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an `input_shape`/`
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)
Trainable params: 225,034 (879.04 KB)

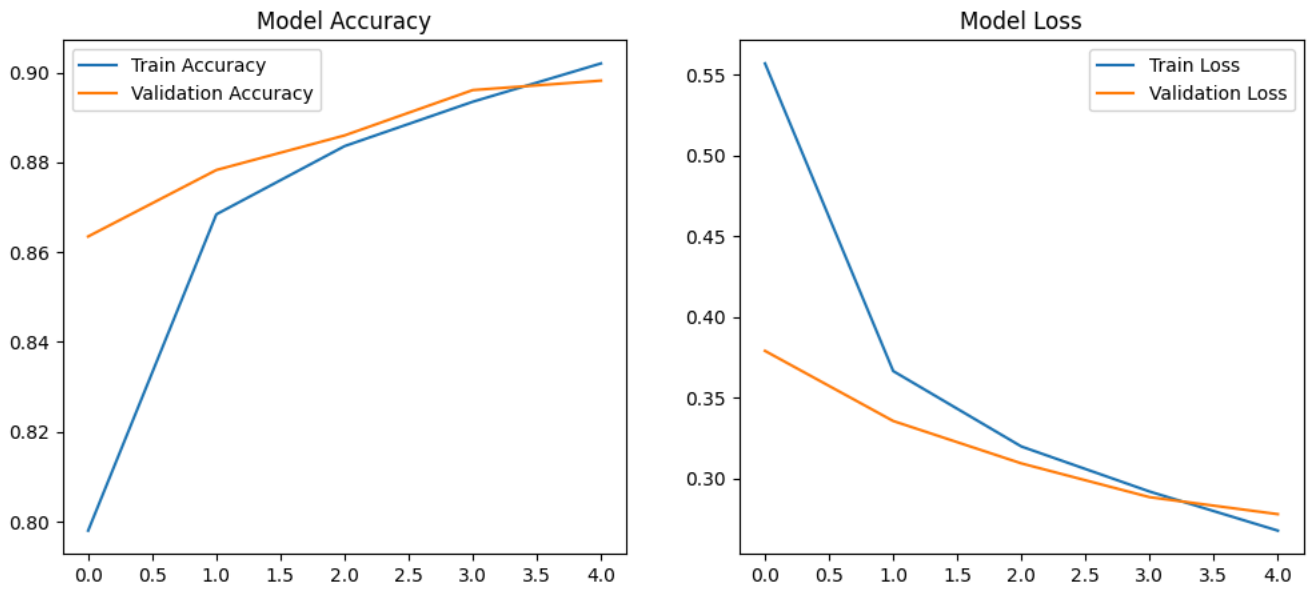
```
history = model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=5, batch_size=64, verbose=1)
```

```
Epoch 1/5
938/938 ━━━━━━━━━━━━━━━━━━━━ 59s 61ms/step - accuracy: 0.7226 - loss: 0.7713 - val_accuracy: 0.8635 - val_loss: 0.3789
Epoch 2/5
938/938 ━━━━━━━━━━━━━━━━━━━━ 75s 54ms/step - accuracy: 0.8633 - loss: 0.3817 - val_accuracy: 0.8783 - val_loss: 0.3355
Epoch 3/5
938/938 ━━━━━━━━━━━━━━━━━━━━ 88s 61ms/step - accuracy: 0.8812 - loss: 0.3280 - val_accuracy: 0.8860 - val_loss: 0.3092
Epoch 4/5
938/938 ━━━━━━━━━━━━━━━━━━━━ 78s 56ms/step - accuracy: 0.8925 - loss: 0.2927 - val_accuracy: 0.8961 - val_loss: 0.2883
Epoch 5/5
938/938 ━━━━━━━━━━━━━━━━━━━━ 79s 53ms/step - accuracy: 0.9009 - loss: 0.2661 - val_accuracy: 0.8982 - val_loss: 0.2778
```

```
test_loss, test_acc = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {test_acc:.4f}")
model.save('fashion_mnist_cnn.keras')
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - accuracy: 0.8958 - loss: 0.2800
Test Accuracy: 0.8982
```

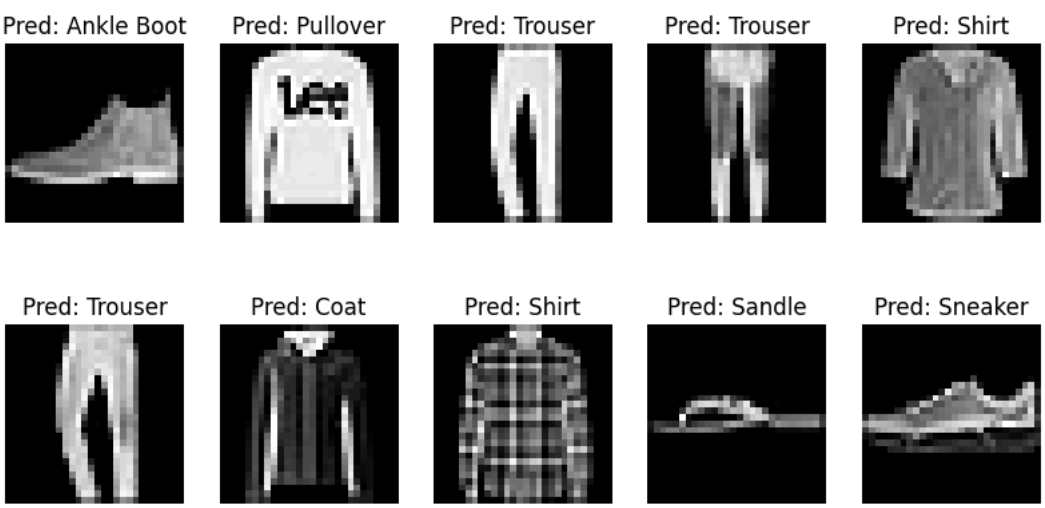
```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.legend()
plt.title('Model Accuracy')

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title('Model Loss')

plt.show()
```



```
predictions = model.predict(X_test)
pred_labels = np.argmax(predictions, axis=1)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step
```

```
plt.figure(figsize=(10, 5))
for i in range(10):
  plt.subplot(2, 5, i+1)
  plt.imshow(X_test[i].reshape(28, 28), cmap='gray')
  plt.title(f"Pred: {class_names[pred_labels[i]]}")
  plt.axis('off')
plt.show()
```



Design and train autoencoder model for image compression

```
import tensorflow as tf
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, UpSampling2D
from tensorflow.keras.models import Model
import numpy as np
import matplotlib.pyplot as plt
```

```
(x_train, _), (x_test, _) = tf.keras.datasets.mnist.load_data()
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
x_train = np.expand_dims(x_train, axis=-1)
x_test = np.expand_dims(x_test, axis=-1)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

```
input_img = Input(shape=(28, 28, 1))

# Encoder
x = Conv2D(32, (3, 3), activation='relu', padding='same')(input_img)
x = MaxPooling2D((2, 2), padding='same')(x)
x = Conv2D(16, (3, 3), activation='relu', padding='same')(x)
x = MaxPooling2D((2, 2), padding='same')(x)
x = Conv2D(8, (3, 3), activation='relu', padding='same')(x)
encoded = MaxPooling2D((2, 2), padding='same')(x)

# Decoder
x = Conv2D(8, (3, 3), activation='relu', padding='same')(encoded)
x = UpSampling2D((2, 2))(x)
x = Conv2D(16, (3, 3), activation='relu', padding='same')(x)
x = UpSampling2D((2, 2))(x)
x = Conv2D(32, (3, 3), activation='relu')(x)
x = UpSampling2D((2, 2))(x)
decoded = Conv2D(1, (3, 3), activation='sigmoid', padding='same')(x)

# Compile the autoencoder
autoencoder = Model(input_img, decoded)
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')
```

```
autoencoder.fit(x_train, x_train, epochs=5, batch_size=128, shuffle=True, validation_data=(x_test, x_test))

decoded_imgs = autoencoder.predict(x_test[:10])
```

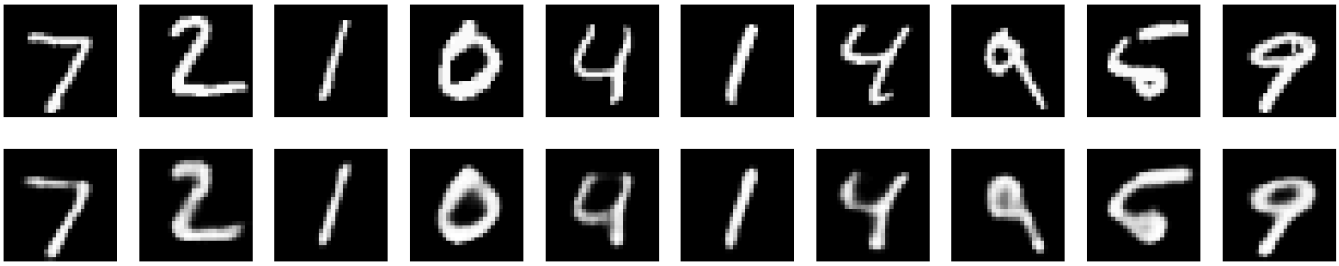
```
Epoch 1/5
469/469 ━━━━━━━━━━━━━━━━━━━━ 113s 233ms/step - loss: 0.2851 - val_loss: 0.1409
Epoch 2/5
469/469 ━━━━━━━━━━━━━━━━━━━━ 136s 221ms/step - loss: 0.1362 - val_loss: 0.1218
Epoch 3/5
469/469 ━━━━━━━━━━━━━━━━━━━━ 143s 223ms/step - loss: 0.1209 - val_loss: 0.1136
Epoch 4/5
469/469 ━━━━━━━━━━━━━━━━━━━━ 140s 219ms/step - loss: 0.1132 - val_loss: 0.1080
Epoch 5/5
469/469 ━━━━━━━━━━━━━━━━━━━━ 101s 216ms/step - loss: 0.1089 - val_loss: 0.1048
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 304ms/step
```

```
# Get compressed (encoded) representations
encoder = Model(inputs=input_img, outputs=encoded)
encoded_imgs = encoder.predict(x_test)

# Print original and compressed shapes for one image
original_shape = x_test[0].shape
compressed_shape = encoded_imgs[0].shape

print("Original image shape:", original_shape)
print("Compressed image shape:", compressed_shape)

# Print actual number of elements (bytes, sort of)
original_size = np.prod(original_shape)
compressed_size = np.prod(compressed_shape)

print("Original image size (pixels):", original_size)
print("Compressed image size (pixels):", compressed_size)
print("Compression ratio: {:.2f}x".format(original_size / compressed_size))
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step
Original image shape: (28, 28, 1)
Compressed image shape: (4, 4, 8)
Original image size (pixels): 784
```

```
Compressed image size (pixels): 128
Compression ratio: 6.12x
```

```
n = 10

plt.figure(figsize=(20, 4))
for i in range(n):
    # Original
    ax = plt.subplot(2, n, i + 1)
    plt.imshow(x_test[i].reshape(28, 28), cmap='gray')
    plt.axis('off')

    # Reconstructed
    ax = plt.subplot(2, n, i + 1 + n)
    plt.imshow(decoded_imgs[i].reshape(28, 28), cmap='gray')
    plt.axis('off')
plt.show()
```



```
encoder = Model(inputs=input_img, outputs=encoded)
encoded_imgs = encoder.predict(x_test)
print("Encoded shape:", encoded_imgs.shape)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step
Encoded shape: (10000, 4, 4, 8)
```

Design and train LSTM model for real world application of your choice

```
import numpy as np
from keras.datasets import imdb
from keras.models import Sequential
from keras.layers import Embedding, LSTM, Dense, Dropout
from keras.preprocessing.sequence import pad_sequences
```

```
vocab_size = 10000
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=vocab_size)
```

```
maxlen = 200
X_train = pad_sequences(X_train, maxlen=maxlen)
X_test = pad_sequences(X_test, maxlen=maxlen)
```

```
model = Sequential([
    Embedding(input_dim=vocab_size, output_dim=128),
    LSTM(64),
    Dropout(0.5),
    Dense(1, activation='sigmoid')
])

model.build(input_shape=(None, maxlen))
model.summary()
```

```
Model: "sequential_7"
```

Layer (type)	Output Shape	Param #
embedding_7 (Embedding)	(None, 200, 128)	1,280,000
lstm_7 (LSTM)	(None, 64)	49,408
dropout_7 (Dropout)	(None, 64)	0
dense_7 (Dense)	(None, 1)	65

```
Total params: 1,329,473 (5.07 MB)
Trainable params: 1,329,473 (5.07 MB)
Non-trainable params: 0 (0.00 B)
```

```
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, epochs=3, batch_size=64, validation_split=0.2)
```

```
Epoch 1/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 89s 276ms/step - accuracy: 0.7020 - loss: 0.5594 - val_accuracy: 0.8500 - val_loss: 0.3631
Epoch 2/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 140s 269ms/step - accuracy: 0.9033 - loss: 0.2586 - val_accuracy: 0.8626 - val_loss: 0.3228
Epoch 3/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 148s 288ms/step - accuracy: 0.9336 - loss: 0.1871 - val_accuracy: 0.8642 - val_loss: 0.3350
<keras.src.callbacks.history.History at 0x7a54138a4150>
```

```
# history = model.fit(
#     X_train, y_train,
#     epochs=5,
#     batch_size=64,
#     validation_split=0.2
# )
```

```
loss, accuracy = model.evaluate(X_test, y_test)
print(f'Test Accuracy: {accuracy:.4f}')
```

```
782/782 ━━━━━━━━━━━━━━━━━━━━ 28s 36ms/step - accuracy: 0.8619 - loss: 0.3491
Test Accuracy: 0.8628
```

```
word_index = imdb.get_word_index()
reverse_word_index = {value: key for key, value in word_index.items()}

def decode_review(encoded_review):
    return ' '.join([reverse_word_index.get(i - 3, '?') for i in encoded_review])
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb_word_index.json
1641221/1641221 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
```

```
import random

random_idx = random.randint(0, len(X_test) - 1)
print(f"Review index: {random_idx}")
```

```
print(decode_review(X_test[random_idx]))
prediction = model.predict(X_test[random_idx:random_idx+1])[0][0]
sentiment = "Positive" if prediction > 0.5 else "Negative"
print(f"Prediction: {sentiment} ({prediction:.2f})")
```

```
Review index: 6101
becomes the main focus the episode falls apart part is an ? stereotype and his flat performance highlights this flaw his exploitatic
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 51ms/step
Prediction: Negative (0.39)
```